TORONTO METROPOLITAN UNIVERSITY

Faculty of Engineering and Architectural Science
Department of Electrical, Computer, and Biomedical Engineering

RESTFUL ROVER SIMULATION CONTROL SYSTEM

FastAPI Server, WebSocket-Based Real-Time Control, and Cloud Deployment Using Docker and Azure

Course: COE892 – Distributed Cloud Computing
Term: Winter 2026

Submitted by:
Miguel Varela

Submitted to:
Dr. Jaseemuddin

Date:
04/03/2026

Introduction

This lab extends the rover simulation system developed in previous labs by replacing the gRPC-based communication model with a RESTful architecture using FastAPI. The objective of this lab is to simulate the interaction between an operator and a rover system through HTTP endpoints, enabling full control over map configuration, mine management, and rover operations.

The system is composed of two main components: a FastAPI server and an operator interface. The server is responsible for managing the system state, including the map, mines, and rover entities, while exposing a set of RESTful endpoints to interact with these components. The operator interacts with the server through a web-based interface implemented using HTML, CSS, and JavaScript, allowing real-time visualization and control of the system.

In addition to REST communication, the system also supports real-time rover control using WebSockets. Finally, the entire application is containerized using Docker and deployed to Microsoft Azure, enabling remote access through a web application.

Part 1: FastAPI Server Implementation

In the first part of the lab, a FastAPI server was developed to manage the rover simulation and expose all required endpoints.
The system maintains three main data structures:

- A map object, storing the width and height of the grid
- A dictionary of mines, where each mine contains an ID, coordinates, and serial number
- A dictionary of rovers, where each rover stores its state, position, direction, command list, and execution history

These structures are stored in memory and updated dynamically based on incoming requests. The server implements all required endpoints grouped into three categories: map, mines, and rovers.

Map Endpoints

The map endpoints allow retrieval and modification of the grid dimensions. The GET request returns the current map configuration along with all mine locations, while the PUT request updates the width and height of the map after validating that the values are positive.

Mine Endpoints

The mine endpoints support full CRUD operations. Mines can be created, retrieved, updated, and deleted. When creating or updating a mine, the server validates that the coordinates are within map boundaries. Each mine is assigned a unique identifier and stored in a dictionary.

Rover Endpoints

The rover endpoints manage rover lifecycle and execution. When a rover is created, it is initialized at position (0,0) facing south, with a list of commands. The system validates that all commands belong to the allowed set (L, R, M, D).

The dispatch endpoint executes the rover's command sequence sequentially. Movement logic updates the rover's position based on its direction, while boundary checks ensure the rover does not move outside the map. If a rover attempts to move while positioned on a mine, it is marked as "Eliminated." If a defuse command is executed on a mine location, the mine is removed from the system.

The server also tracks the rover's execution path and returns it in the response, enabling visualization on the client side.

All endpoints implement proper error handling using HTTP status codes and descriptive messages, ensuring robustness and clarity in communication.

Part 2: Operator Implementation

In the second part of the lab, a web-based operator interface was developed using HTML, CSS, and JavaScript to provide a user-friendly interaction layer for the system.

The interface renders the map as a dynamic grid, where each cell corresponds to a coordinate in the environment. Mines are visually represented on the grid and can be added or removed through direct interaction, allowing the user to modify the environment intuitively. The map dimensions can also be updated dynamically through input controls, which trigger API calls to the backend.

Rover management is handled through a control panel that allows users to create, update, dispatch, and delete rovers. Each rover is displayed with its current status, enabling the operator to monitor system activity in real time.

The interface communicates with the backend using asynchronous HTTP requests, ensuring that updates do not block the user experience. The map and rover list are refreshed periodically, maintaining synchronization with the server state.

Rover movement is visualized directly on the grid, where the execution path is marked and the current position is highlighted. This provides clear feedback on rover behaviour and execution results.

Additionally, a terminal-style log panel displays formatted JSON responses from the server, allowing users to inspect system outputs and debug interactions effectively.

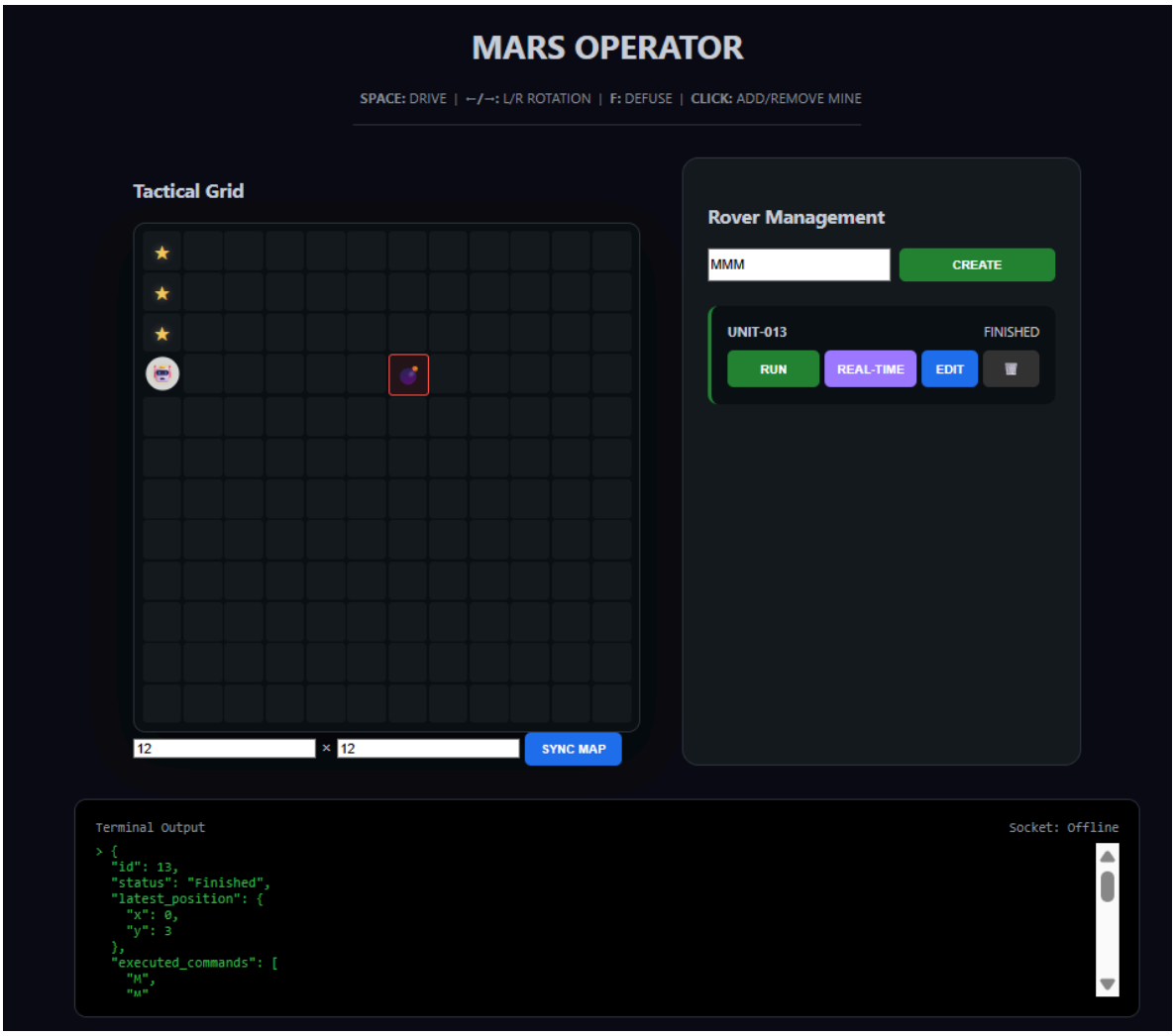

Figure 1 - User Interface of the lab, including the map, logs, and user interaction.

Part 3: WebSocket Real-Time Control (Bonus)

An optional WebSocket endpoint was implemented to enable real-time rover control.

Unlike REST endpoints, which process predefined command sequences, the WebSocket connection allows the operator to send commands interactively, one at a time. This enables fine-grained control over rover behaviour and introduces a real-time interaction model.

Upon establishing a WebSocket connection, the server validates that the rover is in a suitable state (“Not Started” or “Finished”). The rover’s command history is then reset, and its status is updated to “Moving” to indicate active control.

Each command sent by the operator is processed immediately. Movement commands update the rover's position, rotation commands update its direction, and defuse commands remove mines from the map. The server responds to each command with a JSON message containing the updated state, ensuring continuous feedback.

If the rover attempts to move while positioned on a mine, it is immediately marked as "Eliminated," and the connection is terminated. This enforces consistency with the execution rules defined in the system.

The client listens for WebSocket messages and updates the interface in real time, including rover position, path, and status. This feature demonstrates real-time bidirectional communication and enhances the interactivity of the system.

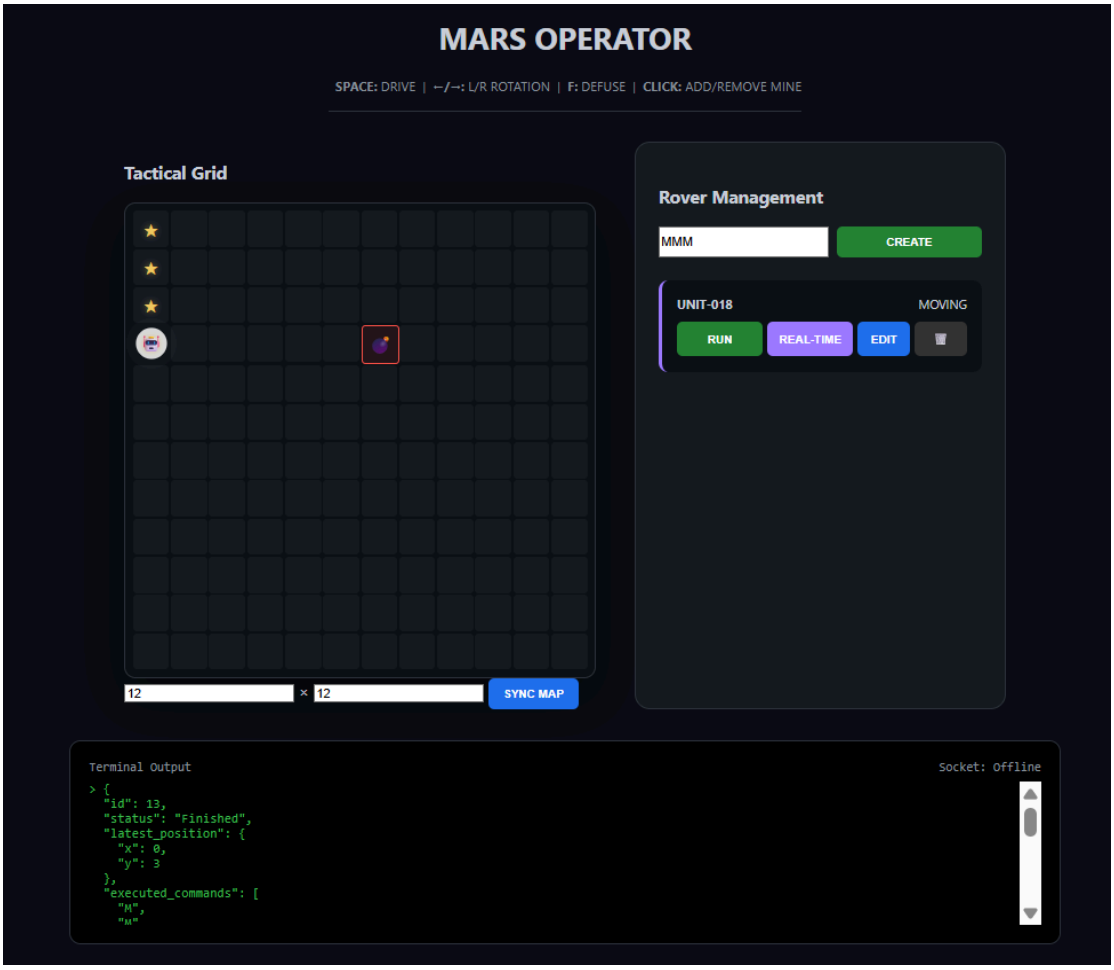

Figure 2 - Real-time interaction is active. Users can move around with the controls and defuse the bomb.

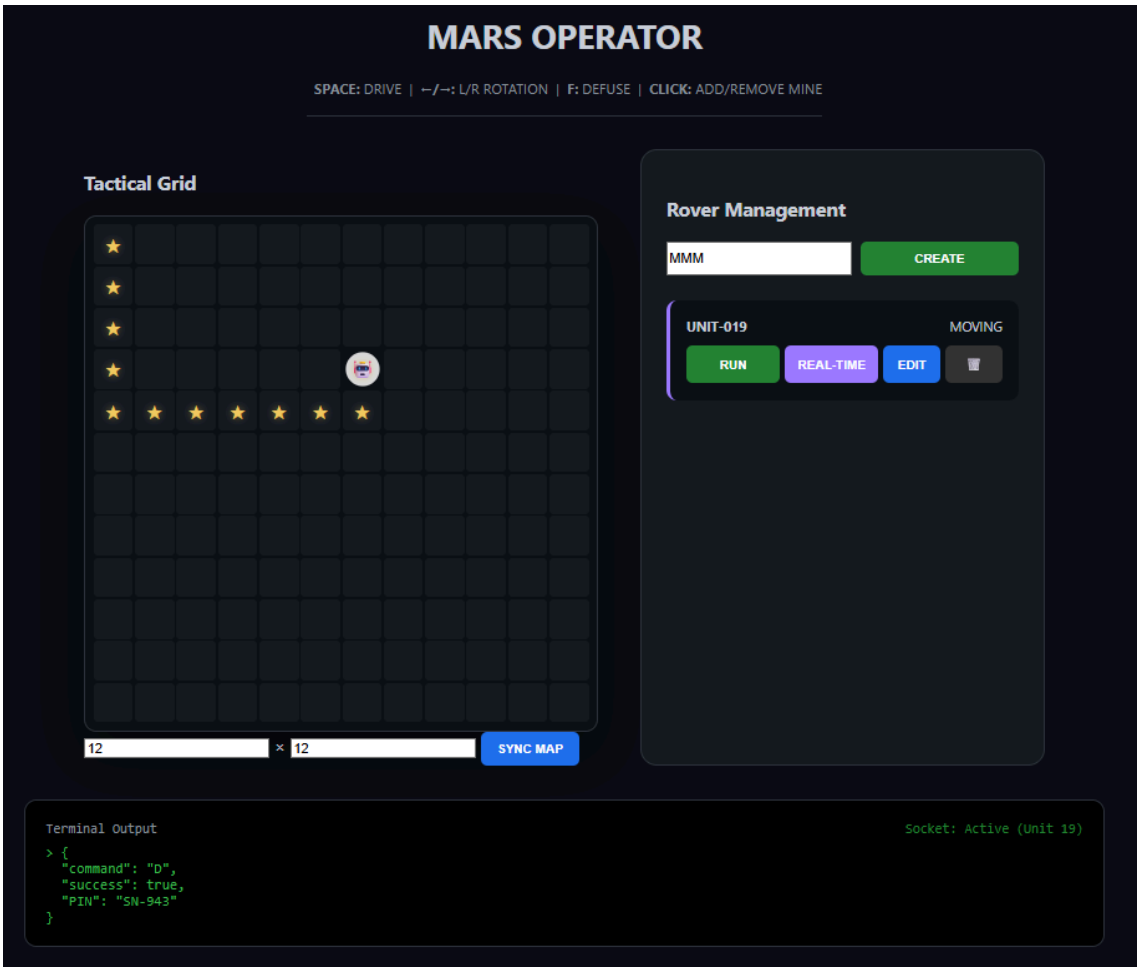

Figure 3 - Bomb defused using real-time interaction.

Part 4: Deployment using Docker and Azure

In the final part of the lab, the application was containerized and deployed to Microsoft Azure to enable remote access.

A Dockerfile was created to package the FastAPI application along with its dependencies. The container exposes port 80 and runs the application using Uvicorn, ensuring compatibility with Azure Web App requirements.

The deployment process involved building the Docker image locally, pushing it to Azure Container Registry, and configuring a Web App for Containers to use the uploaded image. This setup allows the application to be hosted in a scalable and managed cloud environment.

Additional configurations were required to ensure proper functionality, including setting the correct port, enabling WebSockets, and configuring environment variables. These settings ensure that both REST and real-time communication operate correctly in the deployed environment.

The containerized approach ensures consistency between local and cloud environments, simplifying deployment and reducing configuration issues. Once deployed, the application became accessible through a public Azure URL, allowing users to interact with the system remotely through a browser.

Results

The system was tested end-to-end to validate all functionalities. The map was successfully initialized and updated through the operator interface. Mines were added and removed dynamically, and their positions were correctly reflected in the grid.

Rovers were created with custom command sequences and dispatched successfully. During execution, rovers followed the expected movement logic, updated their paths, and handled mine interactions correctly. In cases where a rover encountered a mine without defusing it, the system correctly marked it as eliminated.

The WebSocket implementation allowed real-time control of rovers, with immediate feedback displayed in the interface. Commands were processed correctly, and the system maintained consistent state synchronization between client and server.

The deployment to Azure was successful, and the application was accessible remotely with full functionality, including REST endpoints and WebSocket communication.

Conclusion

This lab introduced a RESTful architecture for controlling a rover simulation system using FastAPI. By transitioning from gRPC to HTTP-based communication, the system demonstrates how modern web APIs can be used to manage application state and interactions effectively.

The implementation highlights key concepts such as API design, input validation, state management, and real-time communication using WebSockets. Additionally, the deployment process provided practical experience with containerization and cloud hosting using Docker and Azure.

Overall, the system successfully integrates backend logic, frontend interaction, and cloud deployment into a cohesive solution. This lab reinforces fundamental concepts in web development and distributed systems, particularly in the areas of scalability, modularity, and system integration.